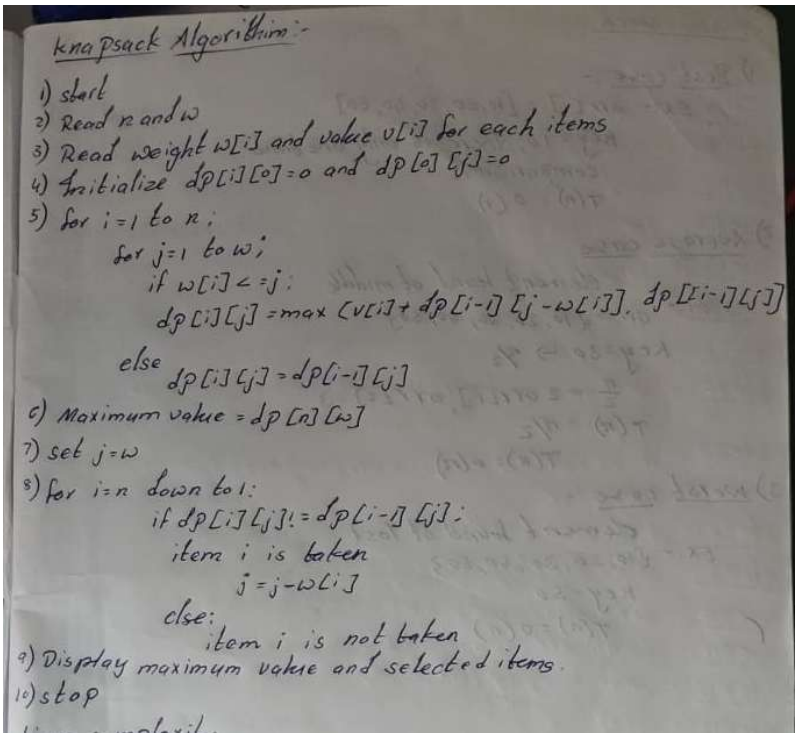
PRACTICAL: 05

AIM:- Implementation of a knapsack problem using dynamic programming.

Algorithms:-



Program/code:-

```
#include<iostream>
using namespace std;
class Knapsack {
private:
```

```
    int w[10];       // weights
    int v[10];       // values
    int n;           // number of items
    int W;           // capacity
    int dp[10][100]; // dp table
public:
    // ── Get Input ──
    void getInput() {
        cout << "Enter number of items   : ";
        cin  >> n;
        cout << "Enter knapsack capacity : ";
        cin  >> W;
        for(int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
            cin  >> w[i] >> v[i];
        }
    }
    // ── Build dp table ──
    void solve() {
        // base case — row 0 and col 0
        for(int i = 0; i <= n; i++) dp[i][0] = 0;
        for(int j = 0; j <= W; j++) dp[0][j] = 0;
        // fill table
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= W; j++) {
```

```
            if(w[i] <= j) {
                // take or not take
                dp[i][j] = max(
                    v[i] + dp[i-1][j-w[i]],
                    dp[i-1][j]
                );
            } else {
                // cannot take item i
                dp[i][j] = dp[i-1][j];
            }
        }
    }
}
// ── Print dp table ──
void printTable() {
    cout << "\nDP Table:" << endl;
    cout << "     ";
    for(int j = 0; j <= W; j++)
        cout << j << "  ";
    cout << endl;
    for(int i = 0; i <= n; i++) {
        cout << "i=" << i << "  ";
        for(int j = 0; j <= W; j++)
            cout << dp[i][j] << "  ";
        cout << endl;
    }
```

```
    }
    // ── Find selected items ──
    void findItems() {
        cout << "\nSelected Items:" << endl;
        int j = W;
        for(int i = n; i >= 1; i--) {
            if(dp[i][j] != dp[i-1][j]) {
                cout << "Item " << i
                    << " (w=" << w[i]
                    << ", v=" << v[i]
                    << ") TAKEN " << endl;
                j = j - w[i];
            } else {
                cout << "Item " << i
                    << " (w=" << w[i]
                    << ", v=" << v[i]
                    << ") NOT taken " << endl;
            }
        }
    }
    // ── Show result ──
    void showResult() {
        cout << "\nMaximum Value = "
            << dp[n][W] << endl;
    }
};
```

```
// ── MAIN ──
int main() {
    Knapsack k;
    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();
    return 0;
}
```

Output:-

```
Enter number of items   : 5
Enter knapsack capacity : 9
Item 1 weight and value : 7
6
Item 2 weight and value : 2
1
Item 3 weight and value : 7
7
Item 4 weight and value : 4
2
Item 5 weight and value : 6
3

DP Table:
      0  1  2  3  4  5  6  7  8  9
i=0   0  0  0  0  0  0  0  0  0  0
i=1   0  0  0  0  0  0  0  6  6  6
i=2   0  0  1  1  1  1  1  6  6  7
i=3   0  0  1  1  1  1  1  7  7  8
i=4   0  0  1  1  2  2  3  7  7  8
i=5   0  0  1  1  2  2  3  7  7  8

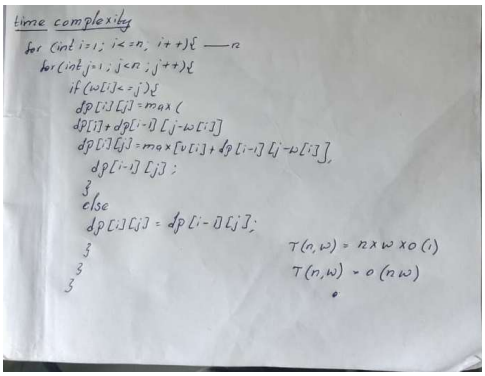
Maximum Value = 8

Selected Items:
Item 5 (w=6, v=3) NOT taken
Item 4 (w=4, v=2) NOT taken
Item 3 (w=7, v=7) TAKEN
Item 2 (w=2, v=1) TAKEN
Item 1 (w=7, v=6) NOT taken

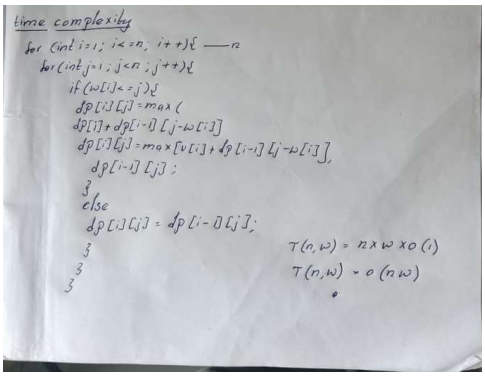

...Program finished with exit code 0
```

Timecomplexity:-

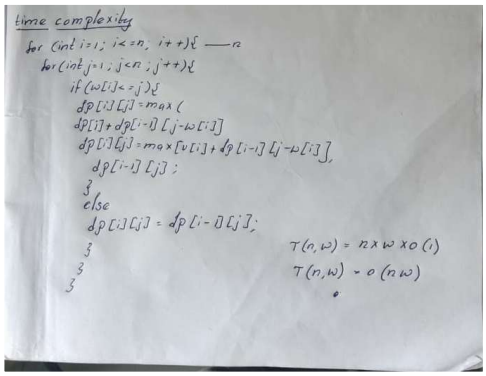
Best case:-



Average Case:-



Worst Case:-



Conclusion:-

The implementation of the 0/1 Knapsack problem using dynamic programming efficiently solves the problem by breaking it into smaller overlapping subproblems and storing intermediate results to avoid redundant computations. This approach reduces the time complexity from the exponential $O(2^n)$ of the brute-force method to $O(n \times W)$, where n is the number of items and W is the knapsack capacity. It ensures optimal selection of items to maximize value without exceeding capacity, making it highly suitable for resource allocation, budgeting, and optimization tasks in real-world applications.